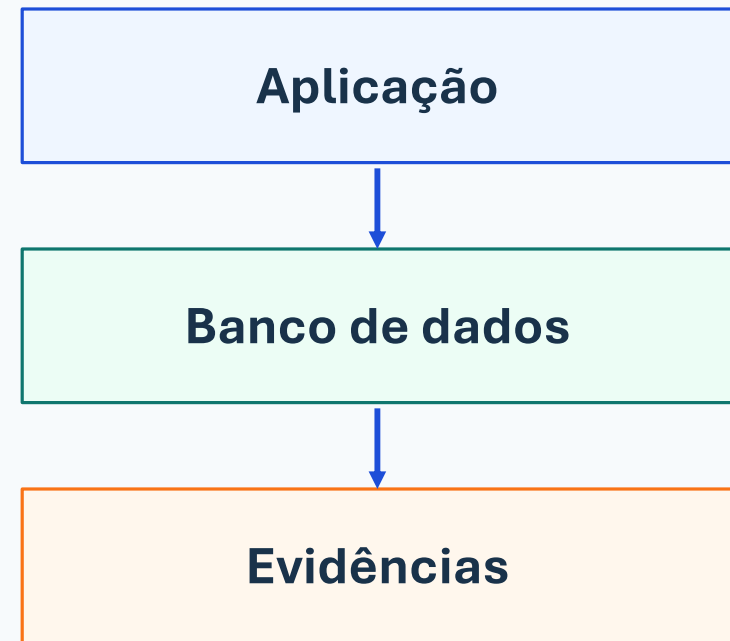


Avaliação de desempenho de aplicações e bancos de dados

Benchmarks e profiling

Disciplina: Avaliação e desempenho de sistemas computacionais



Objetivos da aula

Tópico: O que vamos aprender

Compreender como avaliar aplicações e bancos de dados usando métricas, cargas de trabalho, benchmarks e profiling. A proposta é sair da ideia genérica de “sistema lento” e aprender a produzir evidências úteis para diagnóstico, comparação e melhoria.

Conceitos

Métricas

Benchmark

Profiling

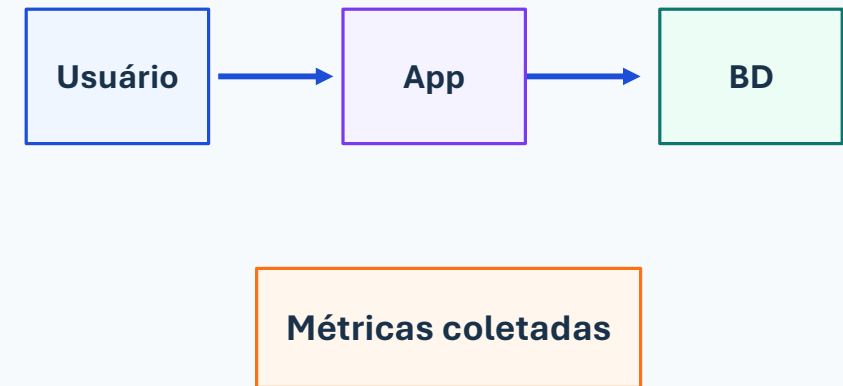
Prática

Leitura sugerida: [Apache JMeter User Manual](#)

Por que avaliar desempenho?

Tópico: Decisão técnica baseada em dados

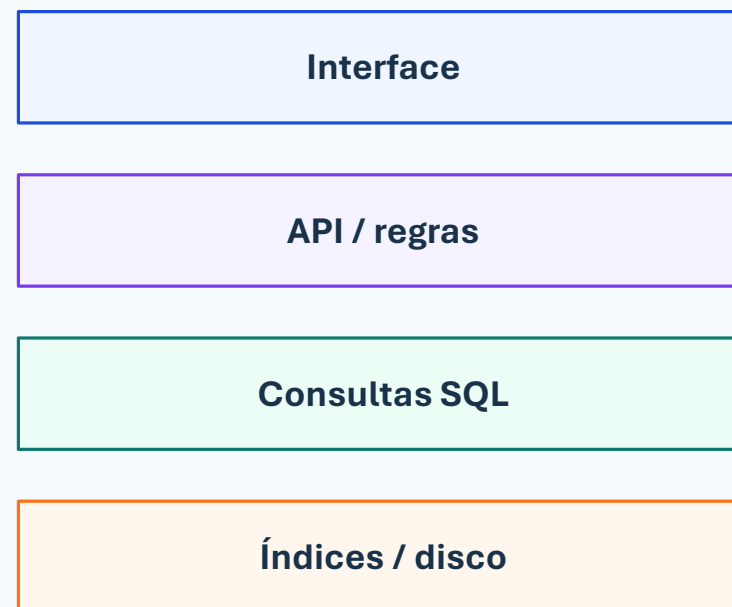
Avaliar desempenho significa transformar observações em evidências. Em aplicações e bancos de dados, isso ajuda a descobrir gargalos, comparar versões, validar melhorias, prever capacidade e evitar decisões baseadas apenas em impressão subjetiva. O objetivo não é “ganhar números”, mas entender o comportamento do sistema.



Sistema sob avaliação

Tópico: Aplicação, banco e ambiente

Uma aplicação não funciona sozinha. Ela depende de interface, código, rede, servidor, banco de dados, cache, armazenamento e configuração. Por isso, uma avaliação séria precisa delimitar o sistema observado e explicar quais componentes fazem parte do experimento e quais ficarão fora da análise.



Métricas principais

Tópico: O que medir

As métricas traduzem o desempenho em valores comparáveis. Em aplicações, usamos tempo de resposta, latência, vazão, taxa de erro e uso de recursos. Em bancos de dados, também observamos consultas por segundo, transações por segundo, tempo de execução, leituras de disco e impacto de índices.

120 ms

tempo de resposta

90 req/s

vazão

0,4%

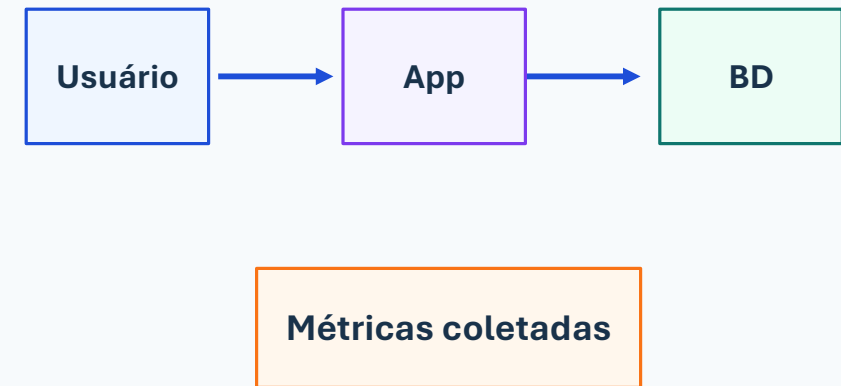
erros

A pergunta correta define a métrica correta

Tempo de resposta

Tópico: Experiência percebida pelo usuário

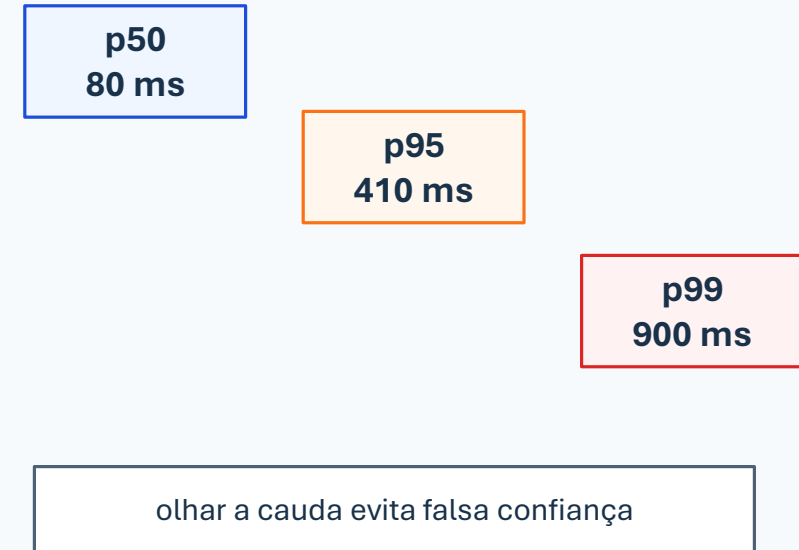
Tempo de resposta é o intervalo entre o pedido do usuário e a chegada da resposta. Em uma aplicação web, pode incluir rede, processamento no servidor, acesso ao banco e renderização no navegador. Esse indicador é muito importante porque costuma refletir diretamente a percepção de lentidão.



Latência e vazão

Tópico: Dois olhares complementares

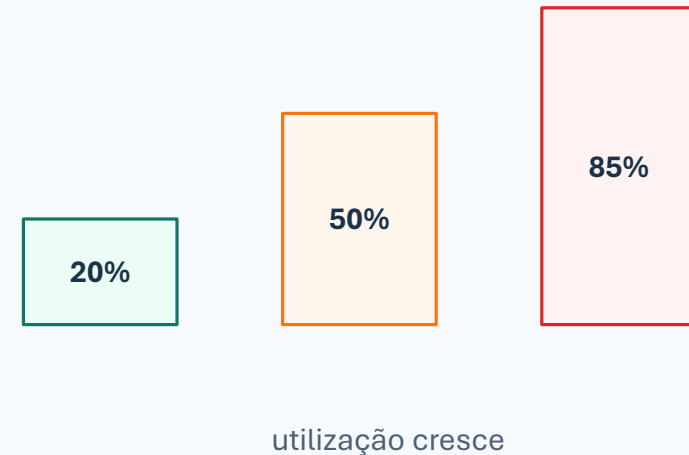
Latência indica o atraso de uma operação individual. Vazão indica quantas operações são concluídas em determinado período. Um sistema pode ter boa vazão média e, ainda assim, oferecer experiência ruim para parte dos usuários se algumas requisições apresentarem latência muito alta.



Utilização de recursos

Tópico: CPU, memória, disco e rede

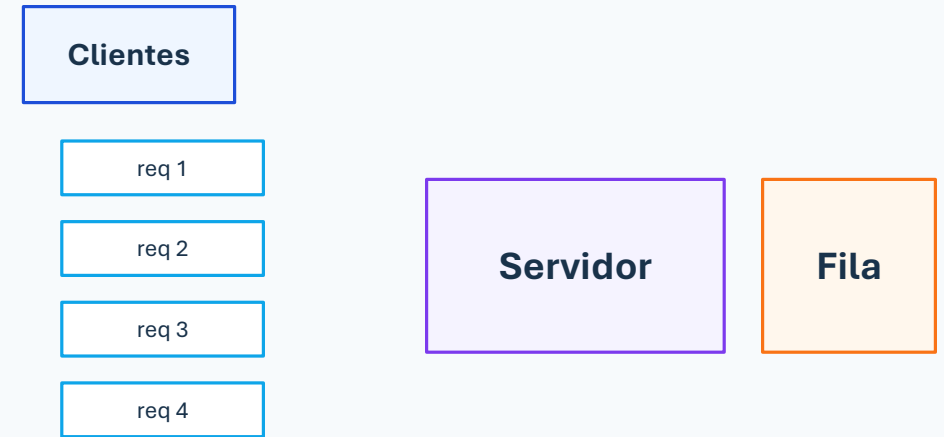
Utilização representa a fração de tempo em que um recurso fica ocupado. Ela ajuda a identificar saturação, mas não deve ser interpretada sozinha. CPU alta pode ser normal em processamento intensivo, enquanto disco alto pode indicar consultas sem índice ou excesso de leitura desnecessária.



Carga de trabalho

Tópico: Representatividade do experimento

Carga de trabalho é o conjunto de requisições submetidas ao sistema. Ela precisa representar o uso real: quantidade de usuários, tipos de operações, ritmo de chegada, dados acessados e duração do teste. Uma carga mal escolhida pode produzir resultados aparentemente corretos, mas inúteis.

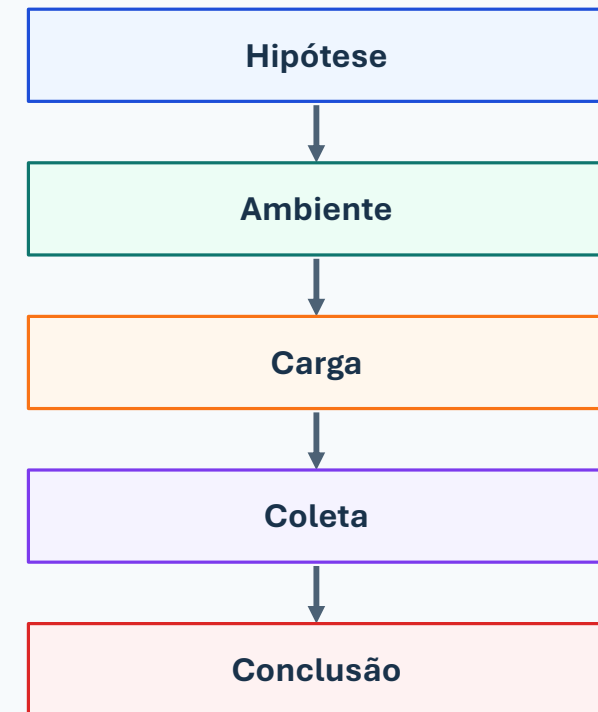


carga = requisições + perfil + ritmo

Parâmetros e variáveis

Tópico: O que controlamos no teste

Parâmetros são elementos que variamos para observar o efeito no desempenho: número de usuários, tamanho da base, índices, versão do software, configuração do servidor ou tipo de consulta. Variáveis não controladas devem ser registradas, pois podem interferir no resultado final.



Medição, benchmark e profiling

Tópico: Três práticas diferentes

Medição é a coleta de dados reais. Benchmark é um teste padronizado ou planejado para comparar desempenho sob uma carga conhecida. Profiling é a investigação interna do comportamento da aplicação, indicando quais funções, consultas ou trechos consomem mais tempo e recursos.



**Mesma carga
mesmo método**

comparação justa

Benchmark

Tópico: Comparação justa

Um benchmark só é útil quando há método. É necessário definir objetivo, ambiente, versão dos componentes, massa de dados, carga, duração, aquecimento, repetição e critério de comparação. Sem esse cuidado, o teste vira apenas uma demonstração momentânea, não uma evidência técnica confiável.

Leitura sugerida: [k6 Documentation](#)



**Mesma carga
mesmo método**

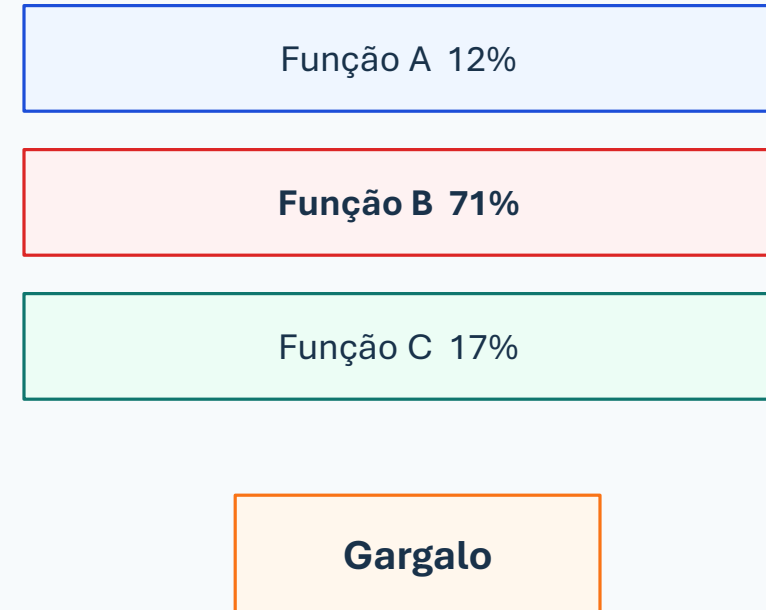
comparação justa

Profiling

Tópico: Localizando onde o tempo é gasto

Profiling ajuda a responder por que o sistema está lento. Em vez de medir apenas o tempo total, ele detalha chamadas, funções, consultas ou eventos. Assim, o avaliador identifica trechos prioritários para otimização e evita gastar esforço em partes que quase não afetam o resultado.

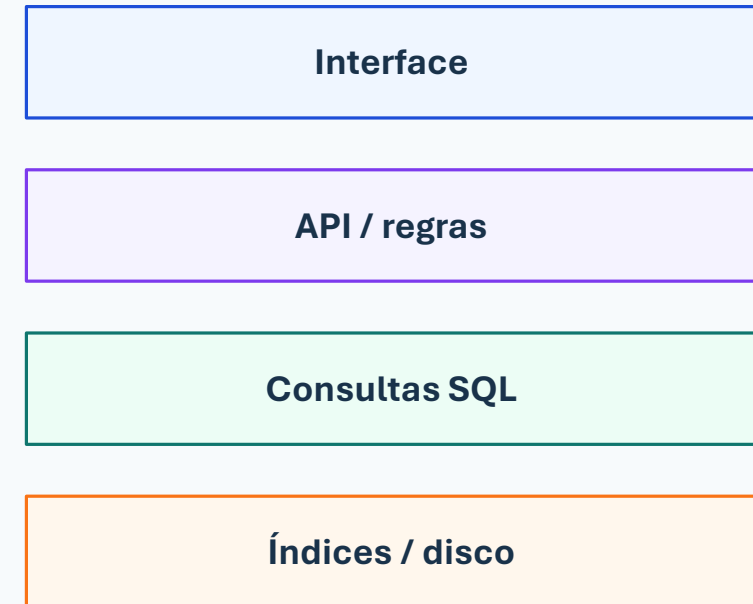
Leitura sugerida: [Python cProfile](#)



Gargalos em aplicações

Tópico: Onde procurar primeiro

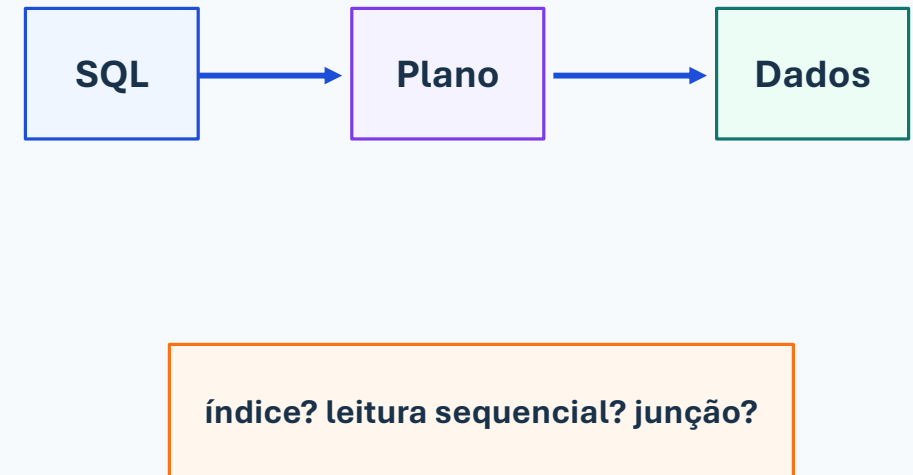
Gargalo é o ponto que limita o desempenho do sistema. Ele pode estar no código, no banco, na rede, no disco, em chamadas externas, na memória ou na configuração. A avaliação busca descobrir qual recurso está limitando a entrega antes de propor qualquer melhoria.



Banco de dados no caminho crítico

Tópico: Consultas e transações

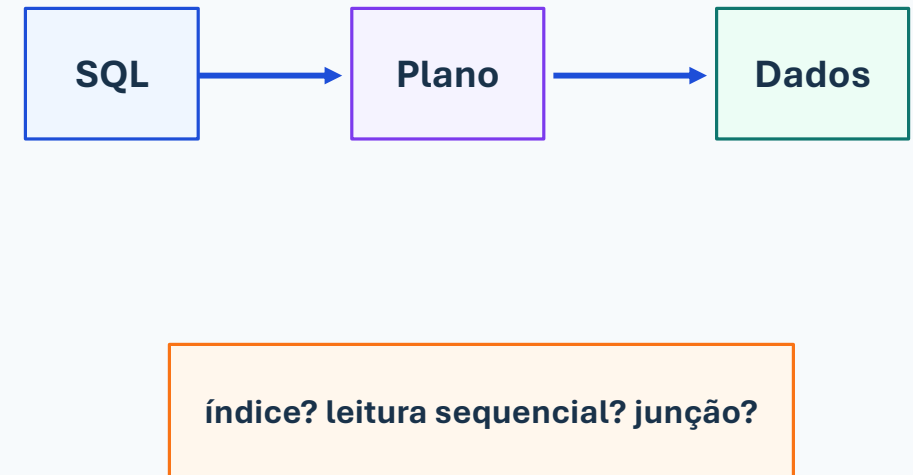
Em muitos sistemas, o banco de dados está no caminho crítico da requisição. Uma consulta lenta, um índice ausente, bloqueios entre transações ou leitura excessiva podem aumentar muito o tempo de resposta. Por isso, avaliar aplicação sem observar o banco pode esconder a causa real.



Plano de execução SQL

Tópico: Como o banco decide executar

O plano de execução mostra como o banco pretende localizar, filtrar, ordenar e combinar dados. Comandos como EXPLAIN e EXPLAIN ANALYZE ajudam a entender se a consulta usa índice, faz varredura completa, realiza junções caras ou consome muitos blocos de leitura.

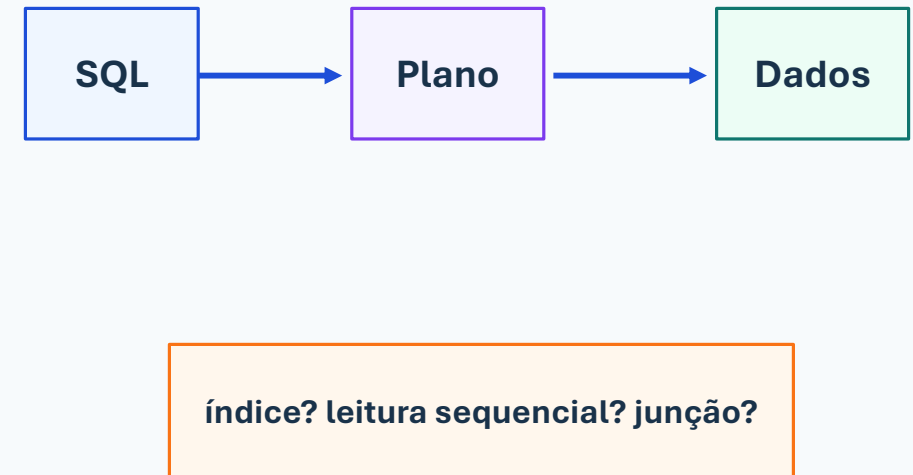


Leitura sugerida: [PostgreSQL EXPLAIN](#)

Índices e desempenho

Tópico: Acesso mais eficiente aos dados

Índices podem acelerar consultas porque reduzem a quantidade de linhas examinadas. Porém, eles também ocupam espaço e tornam inserções, atualizações e exclusões mais custosas. Avaliar desempenho exige verificar se o índice melhora a consulta sem prejudicar indevidamente outras operações.



Transações por segundo

Tópico: Desempenho de bancos de dados

Em bancos de dados, uma métrica comum é TPS, ou transações por segundo. Ela indica quantas transações completas o sistema consegue concluir em determinado tempo. TPS deve ser interpretado junto com latência, erros, bloqueios, tamanho da base e perfil das operações executadas.

$$U = X \times S$$

$$N = \lambda \times R$$

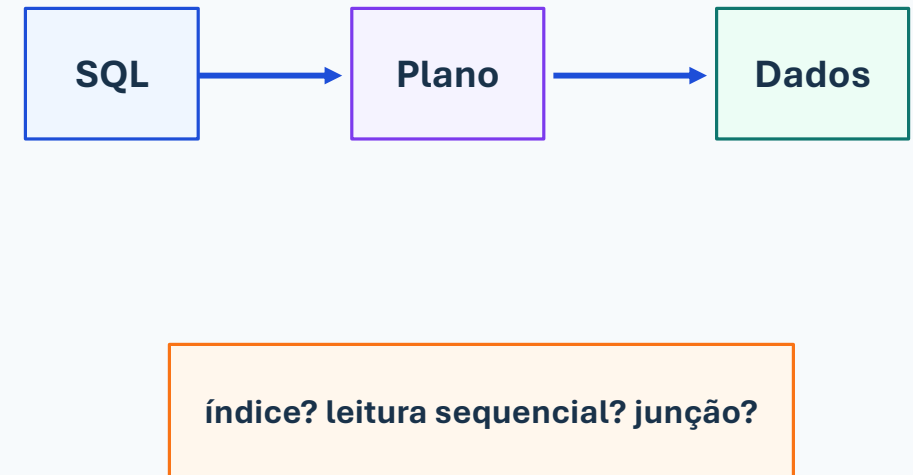
$$\text{TPS} = \text{transações/s}$$

Leitura sugerida: [pgbench PostgreSQL](#)

Profiling de consultas

Tópico: EXPLAIN ANALYZE na prática

Para investigar uma consulta lenta, execute primeiro a consulta isolada, depois analise o plano com EXPLAIN ou EXPLAIN ANALYZE. Observe tempo total, linhas estimadas, linhas reais, tipo de varredura e uso de índices. A diferença entre estimativa e realidade pode indicar estatísticas desatualizadas.



Ferramentas gratuitas

Tópico: Aplicações acadêmicas possíveis

Para aulas práticas, é possível usar ferramentas gratuitas e amplamente documentadas. JMeter, k6 e Locust ajudam em testes de carga; PostgreSQL, SQLite e pgbench ajudam em experimentos com banco; cProfile e Chrome DevTools ajudam a investigar execução e gargalos em código e navegador.

Leitura sugerida: [Locust Documentation](#)

JMeter

k6

Locust

pgbench

cProfile

gratuitas / acadêmicas

JMeter

Tópico: Teste de carga com interface gráfica

O Apache JMeter é muito usado para criar planos de teste com grupos de usuários, requisições HTTP, temporizadores, verificações e relatórios. Por ter interface gráfica, pode ser didático para visualizar a estrutura do experimento, embora também exija cuidado para não criar carga artificialmente irreal.

Leitura sugerida: [Apache JMeter](#)

JMeter

k6

Locust

pgbench

cProfile

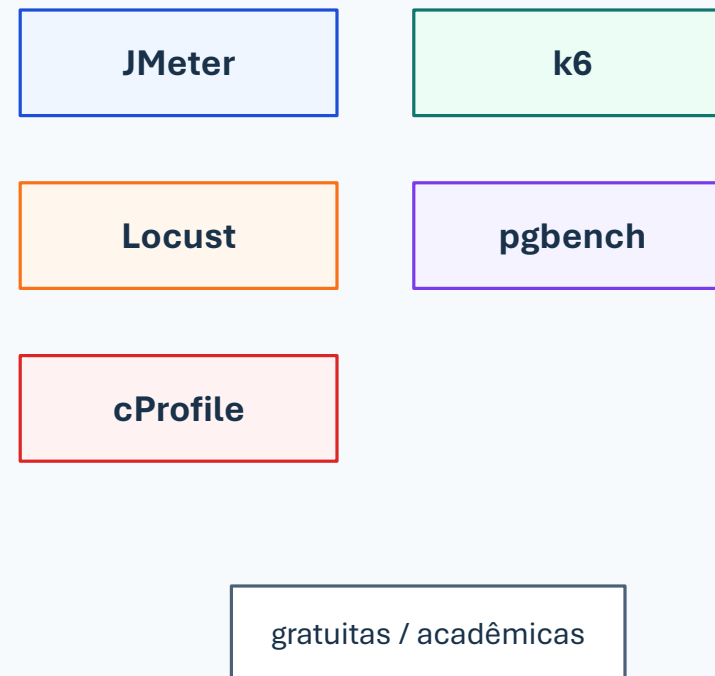
gratuitas / acadêmicas

k6

Tópico: Teste de carga por script

O k6 permite escrever testes em JavaScript, executar pelo terminal e definir métricas, checks e thresholds. Esse formato é interessante para versionar o experimento junto ao projeto, repetir testes em laboratório e comparar resultados antes e depois de uma alteração na aplicação.

Leitura sugerida: [k6 thresholds](#)



Locust

Tópico: Usuários simulados em Python

O Locust descreve o comportamento dos usuários em código Python. Em vez de apenas disparar requisições soltas, é possível simular fluxos, como acessar página inicial, pesquisar, abrir detalhes e enviar formulário. Isso aproxima o benchmark de um comportamento real de uso.

[Leitura sugerida: Locust](#)

JMeter

k6

Locust

pgbench

cProfile

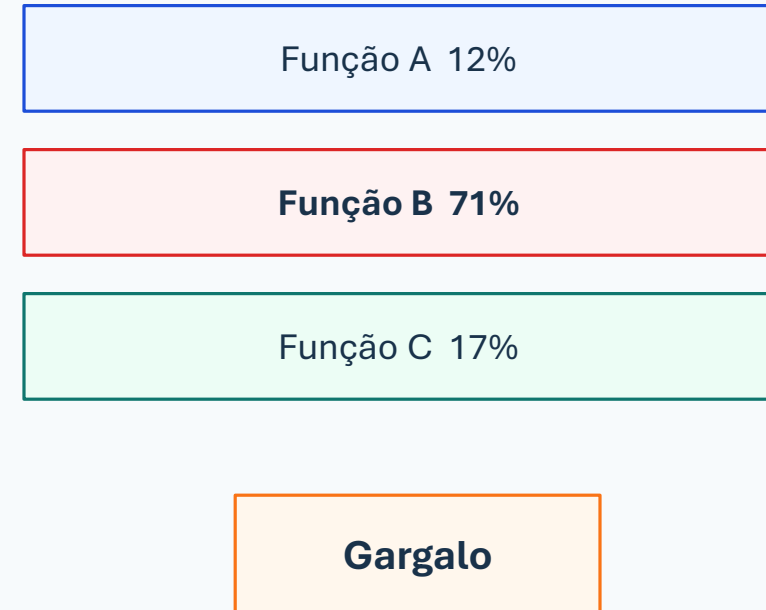
gratuitas / acadêmicas

Chrome DevTools

Tópico: Profiling no navegador

Nem todo problema de desempenho está no servidor. O navegador pode gastar tempo com JavaScript, layout, renderização, imagens e requisições. O painel Performance do Chrome DevTools permite registrar uma execução e observar eventos que afetam a experiência do usuário na interface.

Leitura sugerida: [Chrome DevTools Performance](#)

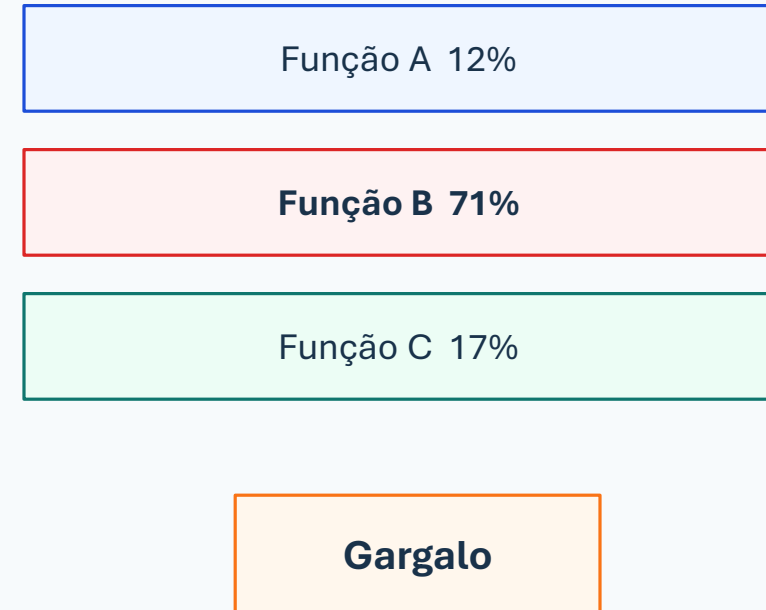


Python cProfile

Tópico: Profiling de código

O cProfile registra quantas vezes funções foram chamadas e quanto tempo foi gasto nelas. Em uma aplicação didática, ele pode mostrar se o gargalo está em cálculo, leitura de arquivo, manipulação de dados ou chamada ao banco. O resultado orienta a próxima etapa de otimização.

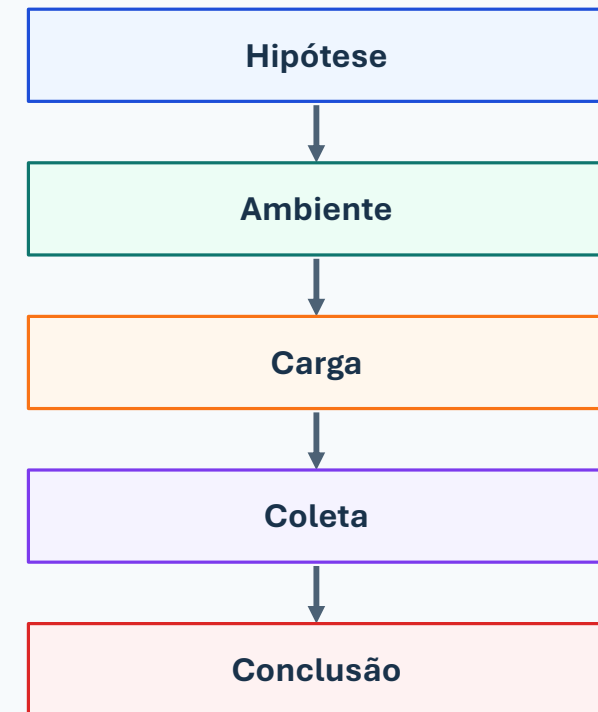
Leitura sugerida: [Python profilers](#)



Passo a passo 1

Tópico: Planejar o experimento

Antes de executar qualquer ferramenta, escreva o objetivo do teste: qual sistema será avaliado, qual hipótese será verificada, quais métricas serão coletadas, qual carga será usada e qual resultado será considerado aceitável. Esse planejamento evita testes improvisados e conclusões frágeis.



Passo a passo 2

Tópico: Preparar ambiente e versão

Registre o ambiente usado: máquina, sistema operacional, versão da aplicação, versão do banco, volume de dados, configurações relevantes e horário do teste. Em avaliações simples, esse registro já aumenta muito a qualidade do relatório e facilita repetir o experimento posteriormente.

✓ Repetir

✓ Isolar variáveis

✓ Registrar versão

✓ Guardar scripts

✓ Comparar p95

Passo a passo 3

Tópico: Executar benchmark HTTP com k6

Crie um arquivo teste.js, defina usuários virtuais e duração, execute k6 run teste.js e observe métricas como duração das requisições, taxa de falhas e requisições por segundo. Para a aula, use uma aplicação local ou endpoint de teste autorizado, nunca sistemas de terceiros sem permissão.

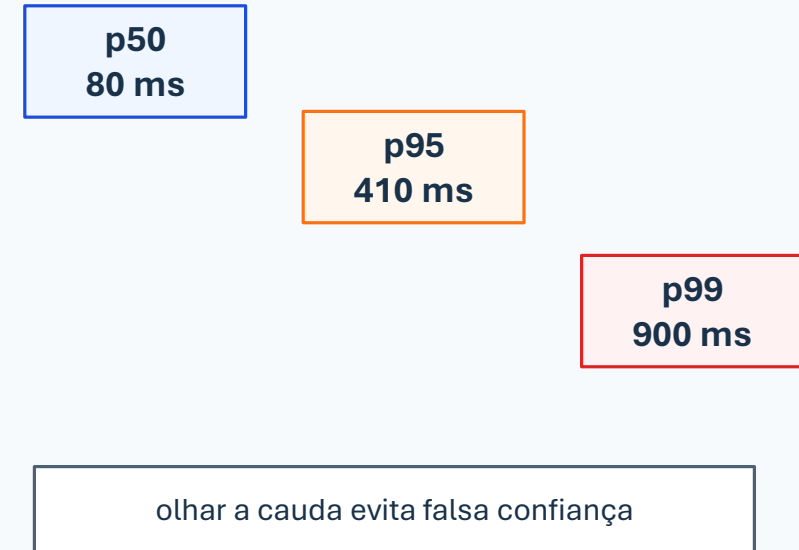
[Documentação: primeiro teste com k6](#)



Passo a passo 4

Tópico: Interpretar saída do benchmark

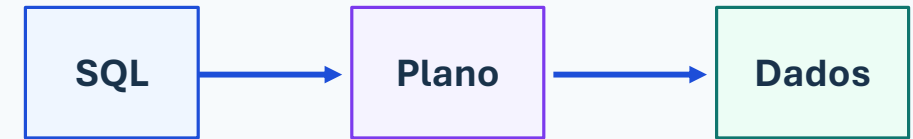
Após executar o benchmark, não observe apenas a média. Analise mínimo, máximo, p90, p95, taxa de erro e vazão. Compare esses valores com o objetivo do teste. Uma média aceitável pode esconder poucos usuários com tempos muito altos, especialmente em momentos de saturação.



Passo a passo 5

Tópico: Banco de dados com SQLite ou PostgreSQL

Para prática em laboratório, crie uma tabela de exemplo com muitos registros, execute uma consulta filtrando por uma coluna, observe o plano de execução, crie um índice e repita a consulta. O objetivo é comparar evidências antes e depois da mudança.



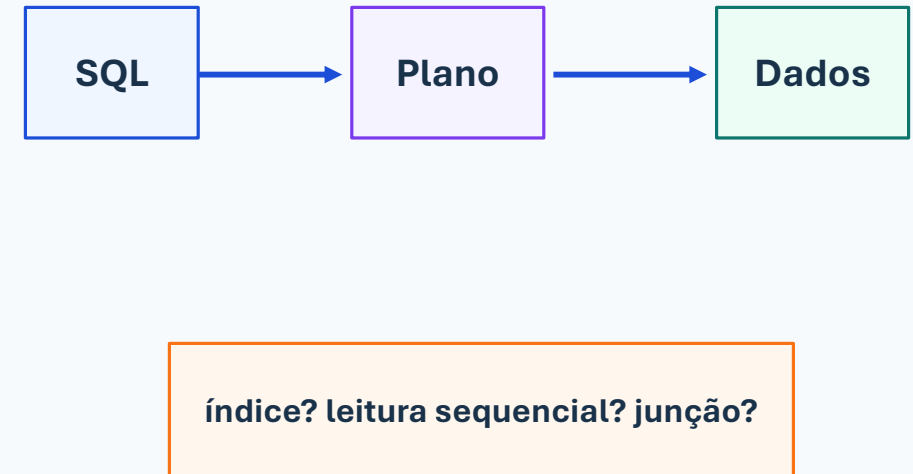
índice? leitura sequencial? junção?

Leitura sugerida: [SQLite EXPLAIN QUERY PLAN](#)

Passo a passo 6

Tópico: Comandos SQL para demonstração

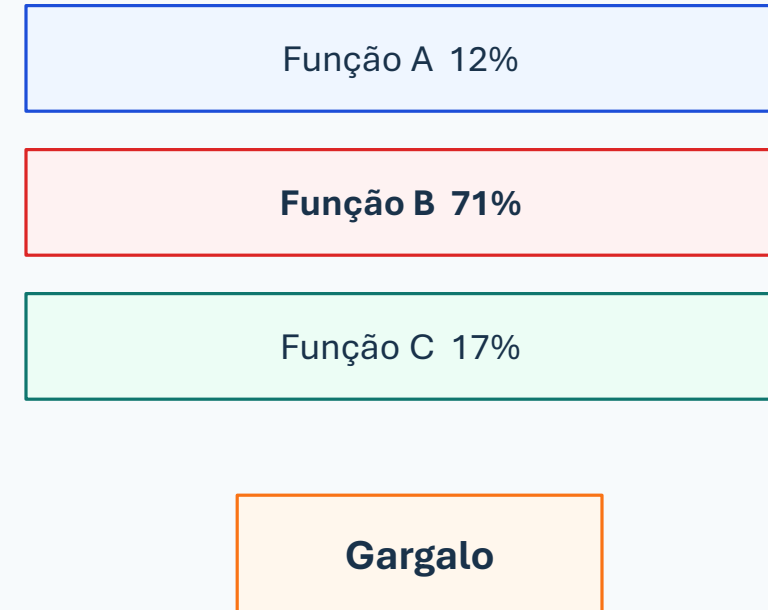
Um roteiro simples pode usar CREATE TABLE, INSERT, SELECT, EXPLAIN QUERY PLAN e CREATE INDEX. Primeiro observe uma busca sem índice; depois crie o índice e repita o plano. A melhoria esperada é reduzir varreduras desnecessárias e aproximar a consulta do acesso direto.



Passo a passo 7

Tópico: Profiling de aplicação

Depois do benchmark indicar lentidão, use profiling para localizar a causa. Registre uma execução representativa, ordene os resultados por tempo acumulado ou tempo total e procure trechos dominantes. Em seguida, formule uma hipótese de otimização e rode novamente o benchmark.



Cuidados estatísticos

Tópico: Repetição e variabilidade

Resultados de desempenho variam por cache, escalonamento, rede, concorrência e ruído do ambiente. Por isso, execute mais de uma rodada, descarte aquecimento quando necessário, registre variações e evite concluir com base em uma única medição isolada. A repetição aumenta a confiança.

✓ Repetir

✓ Isolar variáveis

✓ Registrar versão

✓ Guardar scripts

✓ Comparar p95

Erros comuns

Tópico: O que evitar

Alguns erros comprometem a avaliação: comparar ambientes diferentes, usar massa de dados pequena demais, medir com ferramenta que interfere muito no sistema, ignorar erros, testar em horário inadequado ou mudar várias configurações ao mesmo tempo. Esses problemas tornam a conclusão tecnicamente frágil.

✓ Repetir

✓ Isolar variáveis

✓ Registrar versão

✓ Guardar scripts

✓ Comparar p95

Antes e depois

Tópico: Validando uma melhoria

Otimização só deve ser considerada melhoria quando o resultado melhora em uma métrica relevante sem causar prejuízos importantes em outras. Um índice pode reduzir uma consulta, mas aumentar escrita; cache pode acelerar leitura, mas exigir invalidação correta; paralelismo pode aumentar vazão e também consumo de CPU.



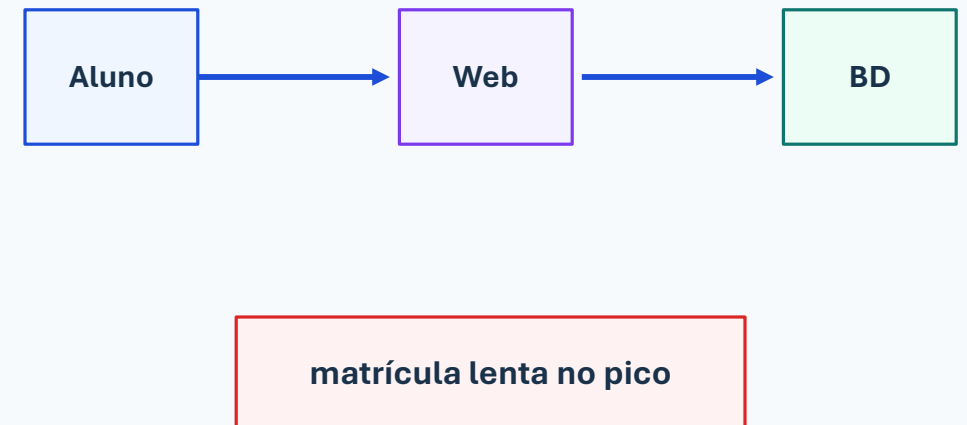
**Mesma carga
mesmo método**

comparação justa

Estudo de caso

Tópico: Sistema acadêmico em período de matrícula

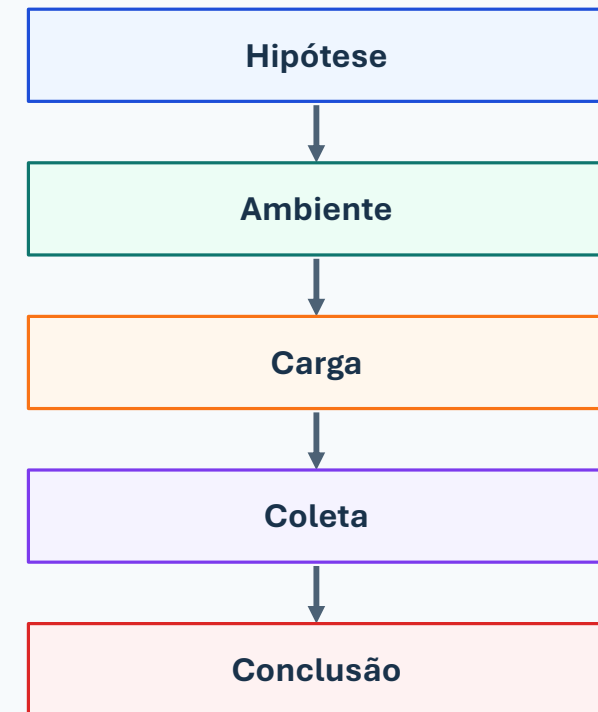
Imagine um sistema acadêmico lento no primeiro dia de matrícula. Os usuários relatam demora para consultar disciplinas e confirmar inscrição. A equipe precisa medir tempo de resposta, erros, vazão, uso de CPU, consultas SQL lentas e concorrência para descobrir se o gargalo está na aplicação ou no banco.



Diagnóstico orientado

Tópico: Sequência de investigação

No estudo de caso, comece pelos logs e métricas gerais. Depois execute benchmark controlado, identifique endpoints lentos, analise consultas envolvidas, use profiling no código e compare resultados antes e depois de ajustes. A investigação deve seguir evidências, não palpites.



Possíveis otimizações

Tópico: Da causa à ação

As soluções dependem da causa identificada. Consulta sem índice pode exigir índice adequado; excesso de chamadas pode exigir cache; processamento pesado pode exigir fila assíncrona; resposta grande pode exigir paginação; bloqueio no banco pode exigir revisão de transações e concorrência.

Interface

API / regras

Consultas SQL

Índices / disco

Ética e segurança

Tópico: Testes com responsabilidade

Testes de carga podem derrubar sistemas, consumir rede e afetar usuários reais. Por isso, benchmarks devem ser feitos com autorização, ambiente controlado, limites definidos e dados fictícios ou protegidos. Avaliação de desempenho também exige responsabilidade com privacidade, disponibilidade e integridade dos serviços.

✓ Repetir

✓ Isolar variáveis

✓ Registrar versão

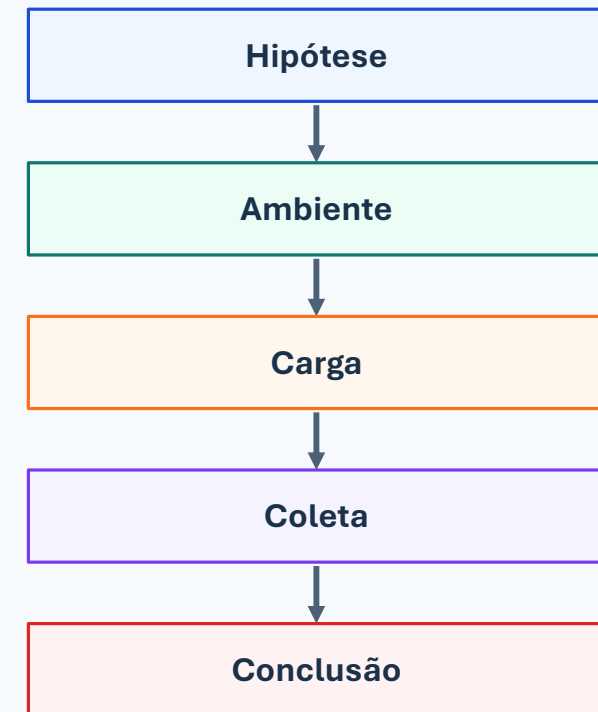
✓ Guardar scripts

✓ Comparar p95

Atividade prática sugerida

Tópico: Roteiro para laboratório

Em duplas, os alunos devem escolher uma aplicação local simples, definir uma métrica principal, executar pequeno benchmark, registrar resultados, localizar um possível gargalo e propor uma melhoria. A entrega deve conter objetivo, ambiente, ferramenta, carga, dados coletados e conclusão justificada.



Modelo de relatório

Tópico: Como apresentar evidências

O relatório deve conter problema, hipótese, ambiente, carga, ferramentas, métricas, procedimento, resultados, gráficos simples, interpretação e recomendação. Uma boa conclusão explica o que os dados indicam, quais limitações existem e quais próximos testes seriam necessários para confirmar a decisão.

✓ Repetir

✓ Isolar variáveis

✓ Registrar versão

✓ Guardar scripts

✓ Comparar p95

Síntese da aula

Tópico: Ideias essenciais

Avaliar desempenho é medir com propósito e interpretar com cuidado. Benchmark compara comportamentos sob carga definida; profiling localiza onde tempo e recursos são consumidos; bancos de dados exigem atenção a consultas, índices, transações e planos de execução. A melhoria deve sempre ser validada por nova medição.

Avaliar → Medir → Interpretar

Otimizar com evidências

Questões para fixação

Tópico: Discussão em sala

1) Por que média de tempo de resposta pode esconder problemas? 2) Quando usar benchmark e quando usar profiling? 3) Como uma consulta sem índice afeta a aplicação? 4) Quais informações mínimas devem aparecer em um relatório de avaliação de desempenho?

✓ Repetir

✓ Isolar variáveis

✓ Registrar versão

✓ Guardar scripts

✓ Comparar p95

Referências e materiais de apoio

JOHNSON, Thienne; COUTINHO, Mauro Margalho. Avaliação de Desempenho de Sistemas Computacionais. LTC, 2011. Material fornecido pelo docente.

Apache JMeter. User Manual. <https://jmeter.apache.org/usermanual/index.html>

Grafana k6 Documentation. <https://grafana.com/docs/k6/latest/>

Locust Documentation. <https://docs.locust.io/> e <https://locust.io/>

PostgreSQL Documentation. Using EXPLAIN. <https://www.postgresql.org/docs/current/using-explain.html>

PostgreSQL Documentation. pgbench. <https://www.postgresql.org/docs/current/pgbench.html>

SQLite Documentation. EXPLAIN QUERY PLAN. <https://www.sqlite.org/eqp.html>

Python Documentation. The Python Profilers. <https://docs.python.org/3/library/profile.html>

Chrome for Developers. DevTools Performance. <https://developer.chrome.com/docs/devtools/performance>